

Session 9 Speaker Script — COMPLETE (Slides 6-14)

06. FEDERATED LEARNING

~8 minutes

WHAT'S ON SCREEN

Train on distributed devices • Privacy preservation • Communication efficiency • Real examples (Apple, Google)

SPEAKING SCRIPT

Federated learning: Train models on user devices without centralizing data.

THE PROBLEM:

Traditional ML: Send all user data to server, train model there.

Issues:

- Privacy: Data exposed during transmission, stored on server
- Bandwidth: Moving terabytes of data is slow and expensive
- Latency: Model training happens on server, takes time
- Regulation: GDPR, CCPA restrict data centralization

Real scenario: Apple wants to improve Siri (voice assistant).

- Billions of users interact with Siri daily
- Generate terabytes of voice data daily
- Traditional ML: Send all voice data to Apple servers, train model
- Problem: Privacy nightmare, bandwidth nightmare, regulatory nightmare

Solution: Federated learning.

HOW IT WORKS:

High level:

1. Model pushed to user's device
2. Model trained locally on user's data (never leaves device)
3. Only model updates sent back to server
4. Server aggregates updates from millions of users
5. Updates merged into new master model
6. Repeat

DETAILED EXAMPLE: APPLE SIRI IMPROVEMENT

Initial: Base Siri model on server. Accuracy 80%.

Step 1: Download phase

- Model (200MB) pushed to 1 million iPhones
- Takes place during device downtime (overnight charging)

Step 2: Local training phase

- Siri runs on-device for a week
- Collects interactions: "Hey Siri, set timer" → user asked for timer
- Model learns: This speech pattern + context = timer command
- After a week: Model has learned from millions of personal interactions
- NEW model kept on device, only improvements (called "delta" or "gradient") sent to server
- Original user data NEVER leaves device

Step 3: Upload phase

- Only model updates (~1MB) sent to server
- Done overnight over WiFi (not cellular, saves bandwidth)
- Updates from 1 million devices arrive at Apple

Step 4: Aggregation phase

- Server-side: Average the 1 million device updates
- Result: New master model that knows what 1 million users want

- New model: Accuracy 81% (1% improvement from learning from real interactions)

Step 5: Repeat

- New model pushed to devices next week
- Process repeats
- Over months: Accuracy improves 80% → 85% → 88% → 90%

PRIVACY GUARANTEES:

End-to-end encryption: Device-to-server communication encrypted. Apple can't see the updates (only the encrypted bytes).

Differential privacy: Add noise to updates. Updates averaged over 1M devices so individual user's data impact invisible.

On-device deletion: User data never leaves device, automatically deleted after training.

Result: Apple has state-of-the-art Siri model, trained on billions of real interactions, but never sees any user data. Privacy preserved.

TECHNICAL CHALLENGES:

Challenge 1: Communication cost

- Sending model updates is expensive
- Solution: Compression (reduce update size), or only send important parts

Challenge 2: Heterogeneity

- Different devices have different capabilities
- iPhone 15: Powerful GPU, fast training
- iPhone 8: Weak processor, slow training
- Solution: Train locally at each device's pace, don't wait for slowest

Challenge 3: Non-IID data

- User A uses Siri for timers, User B for music
- Models trained on different data distributions
- Traditional averaging doesn't work well
- Solution: Advanced aggregation algorithms

Challenge 4: Fault tolerance

- Some devices don't complete training (battery dies, interrupted)
- Can't wait for all updates
- Solution: Async aggregation (aggregate whatever arrives)

REAL EXAMPLES:

APPLE: Siri, Keyboard

- Siri learns from voice commands on-device
- Keyboard learns from typing patterns (emoji predictions, autocorrect)
- Both use federated learning + differential privacy
- Result: Better models, user privacy protected

GOOGLE: GBoard

- Gboard (Google keyboard) learns from typing without sending data
- Federated learning on 500 million devices
- Improved autocorrect, word prediction
- User data never leaves phone

TENSORFLOW FEDERATED:

- Open-source framework from Google
- Researchers can experiment with federated learning
- Companies can deploy their own federated systems

FOR HIRING AI:

Current: Train on centralized resume database

Federated approach: Train on-device

- Company A uploads hiring feedback to edge

- Model trains locally on Company A's data
- Only model updates sent to central server
- Model improves without seeing Company A's confidential hiring data
- Multiple companies contribute, centralized model improves for all
- Privacy-preserving collaborative learning

TRADE-OFFS:

Advantages:

- Privacy: User data stays on-device
- Bandwidth: Only send small model updates, not raw data
- Regulation: Compliant with data protection laws
- Latency: Train during device downtime
- Collaborative: Learn from multiple parties without centralizing data

Disadvantages:

- Complexity: Significantly more complex than centralized
- Slower convergence: Heterogeneous data, slower learning
- Harder to debug: Can't inspect training data
- Device constraints: Complex models don't fit on phones

When to use: When privacy is critical, regulations restrict data centralization, or you have multiple parties wanting to collaborate.

DELIVERY TIPS

Federated learning is increasingly important for privacy. • Apple and Google examples show real scale and impact. • The step-by-step Siri example is concrete. • Privacy-preserving ML is future of AI engineering. • Technical challenges section shows this isn't easy. • For hiring domain: Show how multiple companies could collaboratively learn without sharing confidential data.

07. DIFFERENTIAL PRIVACY

~8 minutes

WHAT'S ON SCREEN

Mathematical privacy guarantees • Epsilon-delta privacy • Adding noise • Queries on sensitive data • Applications

SPEAKING SCRIPT

Differential privacy: Mathematically proven privacy guarantee. Train models on sensitive data while proving no individual's data can be uniquely identified.

THE PROBLEM:

You have sensitive data: Employee salaries, medical records, financial info.

You want to build ML model from it.

Risk: Even if individual records anonymized, model might leak private info.

Example: Hospital wants better disease diagnosis model.

- 10,000 patient medical records (anonymized: no names)

- Train model on all records

- Model is very good (learns from all data)

- But: A researcher could query model: "I have patient with symptoms X, Y, Z. What diagnosis?" - Model might identify specific patient in training data by symptoms

- Researcher now knows that specific patient (now de-anonymized) - Problem: Model leaked private info despite anonymization

Differential privacy: Mathematically guarantee that model doesn't leak any individual's data.

THE GUARANTEE:

"For any two datasets that differ by one record, the model outputs are nearly identical."

Implication: If hospital adds one patient, or removes one patient, model barely changes.

If model barely changes, it means no single person had huge influence on model.

If no single person had influence, can't identify them.

EPSILON-DELTA DEFINITION:

Epsilon (ϵ): Privacy budget. Smaller epsilon = stronger privacy.

Delta (δ): Probability of failing the guarantee. Usually 10^{-5} (0.001% failure rate).

$\epsilon = 0.1$: Very strong privacy, rare to fail. Model severely limited (high noise).

$\epsilon = 1$: Medium privacy, acceptable risk. Some noise, but model still works.

$\epsilon = 10$: Weak privacy, more risk. Little noise, model nearly normal.

HOW IT WORKS:

Process:

1. Train model normally on sensitive data

2. Add calibrated noise to model (amount depends on epsilon)

3. Release noisy model

4. Noise obscures individual records

EXAMPLE: CENSUS DATA

USA Census collects data on millions of people: income, education, race, etc.

Wants to release data publicly for research.

Risk: Even anonymized, researchers could identify individuals.

Solution: Differential privacy.

- Collect all real data
- Add noise to each record (income +/- \$5000 at random)
- Release noisy data
- No

individual can be identified (noise obscures identity)■r/>- Statistical patterns still visible (income distribution looks normal, race distribution looks normal)■r/>- Researchers can study patterns without identifying individuals

Real implementation: US Census Bureau uses differential privacy for 2020 census.

- $\epsilon = 4.75$ (moderate privacy)■r/>- Result: Published data is slightly noisier than ideal, but individuals protected

FOR SENSITIVE DATA IN ML:

Sensitive dataset: 1000 employee salaries

Goal: Build model predicting future salaries (for hiring)

Risk: Model could leak salaries

Differential privacy approach:

1. Train model normally: $Y = 0.5 * \text{experience} + 0.3 * \text{education}$
2. Add noise: $Y = 0.5 * \text{experience} + 0.3 * \text{education} + \text{noise from Laplace distribution}$
3. Release noisy model
4. Model still useful (predicts salary reasonably)■r/>5. But: Individual salaries obscured by noise

TECHNICAL IMPLEMENTATION:

Laplace Mechanism (for numerical queries):

- Query: "What's average salary in dataset?"■r/>- True answer: \$80K
- Add noise: Laplace(0, b) where b calibrated to epsilon
- Noisy answer: \$80.5K or \$79.8K or \$81.2K (roughly \$80K, varies)
- Attacker can't tell if any specific person in dataset changed result■r/>- Noise is calibration of sensitivity (how much one person changes answer)

EXPONENTIAL MECHANISM (for categorical queries):

- Query: "Should we hire this candidate? A, B, C, D, or reject?"■r/>- Calculate score for each option■r/>- Add noise: Don't pick highest score deterministically, pick probabilistically
- Probability weighted by scores but with noise
- Attacker can't invert query to find training data

COMPOSITION:

Privacy budget compounds. If you make 100 queries with $\epsilon = 0.1$ each:■r/>- Total $\epsilon = 100 * 0.1 = 10$ (weakened privacy)■r/>- To maintain strong privacy with 100 queries: Use $\epsilon = 0.01$ per query

- Tradeoff: More queries = more noise needed = worse model

REAL COMPANIES USING DIFFERENTIAL PRIVACY:

LINKEDIN:

- Trains models on employee data (sensitive)■r/>- Differential privacy ensures no individual employee identified
- Can improve models without privacy risk

GOOGLE:

- Chrome data collection: Aggregates user data with differential privacy
- Learns patterns of browser behavior without identifying users

APPLE:

- Combined with federated learning■r/>- Even Apple employees can't reverse-engineer user data
- Maximum privacy guarantee

UBER:

- Driver behavior analysis: Learn patterns without identifying drivers
- Location data with differential privacy■r/>

TRADE-OFFS:

Advantages:

- Formal, proven privacy guarantees (not heuristic)■r/>- Regulatory compliance (satisfies legal requirements)■r/>- User trust (can claim mathematical privacy)■r/>- Enables training on sensitive data safely

Disadvantages:

- Noise reduces model accuracy
- More privacy (smaller epsilon) = more noise = worse model
- Requires careful calibration
- Not always intuitive to practitioners

When to use: When training on sensitive data, when regulations require privacy, when you need formal guarantees.

DELIVERY TIPS

Differential privacy is mathematical—explain carefully. • The epsilon-delta definition is key but can be complex. • Census example is perfect: real-world, government scale. • LinkedIn and other companies show adoption. • The tradeoff (noise vs accuracy) is important. • Federated + differential privacy together is powerful combination.

08. MODEL DISTILLATION & COMPRESSION

~8 minutes

WHAT'S ON SCREEN

Teacher-student learning • Knowledge transfer • Size reduction • Inference speedup • Quantization

SPEAKING SCRIPT

Model distillation: Train a small model to mimic a large model. Transfer knowledge from large (expensive) model to small (cheap) model.

THE PROBLEM:

Large model (100 million parameters):

- Accuracy: 92%
- Inference time: 500ms - Can't run on phone (too big) - Expensive to deploy (server cost)
- Need to deploy AI to billions of users on phones.
- Can't do 500ms inference (user gets frustrated).
- Can't send 100MB model to each device (bandwidth).

Solution: Distillation into small model (1 million parameters).

- Accuracy: 90% (only 2% loss, tiny) - Inference time: 10ms (50x faster!) - Can run on phone - Cheap to deploy

HOW IT WORKS:

Process:

1. Train large model (teacher) normally: 92% accuracy
2. Use teacher model to generate soft labels on training data
 - Traditional: "Is this candidate good? Yes/No (hard label) - Soft label: "Is this candidate good? 92% yes, 8% no (probability)
3. Train small model (student) to match teacher's soft labels
 - Student learns: Teacher thinks this candidate 92% good, I should too
 - Student not just learning classification, learning teacher's reasoning
4. Result: Small model learns from large model

WHY SOFT LABELS MATTER:

Hard labels (traditional): "Candidate good or bad?" Binary.

- Small model learns: This is a hard classification problem
- Hard to compress (needs memorization)

Soft labels (distillation): "92% good, 8% bad."

- Small model learns: Why teacher thinks 92% good (subtle patterns)
- These patterns are compressible
- Small model can learn compact representation

REAL EXAMPLE: GOOGLE BERT COMPRESSION

Original BERT: 340 million parameters. Accuracy 85%.

- Inference: 100ms
- Deployment cost: Huge

Distilled DistilBERT: 40 million parameters (88% size reduction).

- Inference: 10ms (10x faster) - Accuracy: 83.4% (1.6% loss, acceptable)
- Deployment cost: Massively reduced
- Can now run on mobile

HOW TO DO IT (PSEUDOCODE):

...

```

Teacher = trained_large_model()
Student = small_model() // Untrained

for batch in training_data:
    // Teacher generates soft labels
    teacher_output = Teacher(batch) // [0.85, 0.12, 0.03] (probabilities)

    // Student predicts
    student_output = Student(batch) // [0.40, 0.40, 0.20] (initial random)

    // Compare and update student
    loss = KL_divergence(teacher_output, student_output)
    Student.update_weights(loss)
// After training: Student is small but smart

```

TEMPERATURE SCALING:

Temperature (T): Softens probability distributions.

T=1 (normal): [0.9, 0.08, 0.02] (peaked) T=5 (high): [0.25, 0.22, 0.17] (flatter)

Why? High temperature reveals more information.

- Hard probabilities [0.95, 0.05]: Student learns little (obvious what teacher thinks)
- Soft probabilities [0.60, 0.40]: Student learns more (teacher reasons are apparent)
- Used during distillation to make learning richer

QUANTIZATION (Related technique):

Store weights in fewer bits.

Full precision (32-bit floats): Weight = 3.141592653589793

8-bit integer: Weight = 3 (rounded)

- Model size: 4x smaller
- Computation: 4x faster
- Accuracy loss: Minimal (0.5-1%)

INT8 quantization:

- Reduces model size 4x
- Reduces inference time 4x
- Maintains 98% accuracy
- Works great for inference (not training)

PRUNING (Another technique):

Remove unimportant weights.

Neural network has millions of weights. Many are near-zero (don't matter).

- Remove weights < 0.001 in magnitude
- Set them to exactly zero
- Model is now sparse (lots of zeros)
- Can use sparse matrix math (skip zero computations)
- Result: 50% size reduction, 10x speedup, minimal accuracy loss

COMBINATION:

Apply multiple techniques:

1. Distill: 100M → 40M parameters
 2. Quantize: 40M with FP32 → 40M with INT8 (4x size reduction)
 3. Prune: Remove 50% of remaining weights
- Total: 100M → 5M parameters (20x reduction)
- Inference: 500ms → 2.5ms (200x speedup)
- Accuracy: 92% → 88% (small loss)
- Result: Can run on phone in real-time

REAL APPLICATIONS:

GOOGLE PHOTOS:

- Face recognition on billions of photos
- Teacher model (92% accurate, slow)
- Distilled to mobile model (89% accurate, instant)■r/>- Users get instant face grouping on phone

APPLE SIRI:

- Speech recognition model distilled for on-device
- User says something, model responds instantly (no server call)
- Reduces latency, improves privacy

MOBILE BANKING:

- Fraud detection model deployed to banking apps
- Instant fraud checking (no server round-trip)■r/>- Privacy (transaction never leaves your phone)

GAME AI:

- Nintendo uses distilled models for Switch games
- AI opponents run instantly on limited hardware

FOR HIRING AI:

Teacher: Complex model (fine-tuned, with RAG): 95% accuracy, 2 seconds

Student: Distilled model: 92% accuracy, 50ms

On phone: Show recommendations instantly

In hiring portal: Show with higher accuracy, take 2 seconds

TRADE-OFFS:

Distillation advantages:

- Maintains accuracy close to teacher
- Smaller, faster models
- Better than just training small model from scratch (student learns from teacher, not raw data)

Disadvantages:

- Requires trained teacher (extra work)
- Some accuracy loss (unavoidable)
- Hyperparameter tuning (temperature, loss weights)

DELIVERY TIPS

Model compression is increasingly important for edge AI. • BERT example is real and concrete. • Explain soft labels carefully—this is the key insight. • Quantization and pruning are related techniques—show how they stack. • Real applications show practical impact. • For hiring: Show on-device recommendation, instant feedback, privacy benefits.

09. QUANTIZATION & COMPRESSION

~5 minutes

WHAT'S ON SCREEN

Bit-width reduction • Mixed precision • Sparsity • Real-time performance • Hardware support

SPEAKING SCRIPT

Quantization: Reduce precision of numbers to reduce model size and speed up inference.

HOW IT WORKS:

Weights typically stored as float32 (32-bit floating point, 4 bytes each).

- Precision: 7 significant digits (3.141592...)
- Range: 10^{-38} to 10^{38}
- Overkill for inference

INT8 (8-bit integer): 1 byte per weight.

- Range: -128 to 127
- Precision: Limited, but enough for inference
- Size: 4x smaller than float32
- Speed: 4x faster (fewer bytes to move, process)

INT4 (4-bit): 0.5 bytes per weight. Extreme compression.

- Range: -8 to 7
- Precision: Very limited
- Size: 8x smaller than float32
- Usually need fine-tuning to maintain accuracy

MAPPING PROCESS:

float32 value 3.5 in INT8:

- Find range of all weights: -5.2 to 4.8
- Scale factor: $8 / (4.8 - (-5.2)) = 8 / 10 = 0.8$
- Quantize: $3.5 * 0.8 = 2.8 \approx 3$ (as INT8)
- Store as: 3 (1 byte)
- Dequantize: $3 / 0.8 = 3.75$ (close to 3.5)

MIXED PRECISION:

Not all weights need same precision.

- Some weights critical: use INT8
- Some weights less critical: use INT4
- Early layers: Often INT8 (learned carefully)  Later layers: Often INT4 (less sensitive)
- Result: Average bit-width 6, good accuracy

HARDWARE SUPPORT:

Modern AI accelerators support INT8 natively:

- GPUs (NVIDIA with INT8 cores)
- Mobile processors (Apple Neural Engine, Qualcomm)
- TPUs (Google TPU supports INT8, INT4)
- Result: No speed penalty for using INT8

SPARSITY:

Many weights are near-zero (unimportant).

- Normal: Store all million weights
- Sparse: Only store non-zero weights + indices
- 90% sparsity: Only 100k weights stored (10x reduction)

- Sparse math libraries skip zero computations (10x speedup)

ACTIVATION QUANTIZATION:

Not just weights, but also activations (intermediate values).

- Full precision float32 activations: Expensive memory, slow
- INT8 activations: 4x smaller, 4x faster
- Techniques: Calibration (find right INT8 range for each layer)

REAL RESULTS:

ResNet50 (image model):

- Full (FP32): 100MB model, 200ms inference
- INT8 quantization: 25MB model, 50ms inference (4x both)
- Accuracy: 75.8% → 75.4% (tiny loss)

BERT (language model):

- Full: 330MB model, 95ms
- INT8: 83MB model, 24ms (4x reduction)
- Accuracy: 85% → 84.2%

When combined with distillation: 10-20x reduction possible.

DELIVERY TIPS

Quantization is practical and widely adopted. • INT8 is standard for inference. • Mixed precision is advanced technique. • Hardware support shows this is real (not just simulation). • Real results on ResNet and BERT are concrete.

10. REAL-WORLD ADVANCED TECHNIQUES

~8 minutes

WHAT'S ON SCREEN

5 Companies using advanced techniques • Netflix, Google, Apple, LinkedIn, Tesla • Specific techniques and impact

SPEAKING SCRIPT

Let me show you how real companies use these advanced techniques at scale.

EXAMPLE 1: NETFLIX MULTI-ARMED BANDITS

Problem: Netflix has 10,000 titles. Which ones to show in user's feed (personalized rows)?

Traditional approach: Always show most popular or most watched by similar users.

Problem: Miss opportunities. Maybe user would love a hidden gem.

Solution: Multi-armed bandits (Thompson sampling).

- For each user: Maintain probability distribution of "how much will user like title X?"
- Explore: Occasionally show a title with high uncertainty (might love it)
- Exploit: Usually show titles with highest expected rating
- Learn: Each user interaction updates distributions

Algorithm: Contextual Thompson sampling.

- Context: User's genre preferences, watch history, ratings
- For each context: Which title should we show?
- Algorithm picks title that maximizes expected reward + exploration bonus

Impact: Engagement: 5-15% increase in user engagement (more hours watched) Retention: 2-3% reduction in churn (users stay longer) Revenue: 20-30M more hours watched daily = tens of millions in revenue

EXAMPLE 2: GOOGLE ONLINE LEARNING

Problem: Ad bidding landscape changes constantly. Competitors adjust bids, user behavior changes.

Traditional: Update models weekly. Takes 1 week to respond to market changes.

Gap: Advertisers lose money during week when model is stale.

Solution: Online learning.

- Real-time bidding data arrives constantly
- Update model incrementally
- Respond to market changes within hours
- Respond to individual advertiser changes within minutes

Technique: SGD (stochastic gradient descent) on streaming data.

- Each ad impression: Update model slightly
- Billions of impressions per day
- Model improves continuously
- Decay old data (recent data more relevant)

Impact: Bid optimization: 5-10% better ROI for advertisers

- Revenue: Advertisers spend more (getting better ROI)
- Adaptability: Responds to market changes in real-time

EXAMPLE 3: APPLE FEDERATED LEARNING + DIFFERENTIAL PRIVACY

Problem: Improve on-device AI (Siri, Keyboard, Photos) without collecting user data.

Traditional: Send data to servers (privacy nightmare).

Apple's approach: Federated learning on billions of devices.

Process:

1. Model downloaded to device
2. Model trains on-device on user's data
3. Only model updates sent to servers (not raw data)
4. Updates aggregated across devices
5. New model sent back
6. Differential privacy ensures no individual identifiable

Systems using this:

- Siri: Learns from voice commands
- Keyboard: Learns emoji predictions, autocorrect
- Photos: Learns face grouping preferences
- Safari: Learns browsing patterns

Impact: Privacy: Apple truly doesn't see user data Competitive advantage: "Privacy first" marketing message Trust: Users feel safe using Apple products
- Regulation: Compliant with GDPR, CCPA

EXAMPLE 4: LINKEDIN DIFFERENTIAL PRIVACY

Problem: Train recommendation models on sensitive employee/recruiter data. Risk of privacy breach.

Solution: Differential privacy for model training.

Applied to:

- Job recommendation models: Train on who applied where
- Recruiter models: Train on recruiter search patterns
- Profile completion: Recommend what recruiter might add

Technique: Add calibrated noise to gradients during training.

- Epsilon = 1.0 (strong privacy guarantee) Model trained with noise Individual records mathematically unidentifiable

Impact: Privacy: Mathematically provable (passes audits) Trust: Users confident their data is protected

- Regulation: Compliant with labor privacy laws
- Business: Can use sensitive data safely

EXAMPLE 5: TESLA MODEL DISTILLATION

Problem: Autonomous driving AI model too large for car hardware.

- Full model: 500MB, too large for car onboard GPU
- Inference: 2 seconds, too slow (car is moving) Solution: Model distillation + quantization.

Teacher: Large, accurate model (runs on server for training).

- Accuracy: 97% on object detection
- Inference: 200ms (good for training analysis)

Student: Small model for car.

- Distilled from teacher Quantized to INT8 Pruned for sparsity
- Size: 50MB (fits on car) Inference: 20ms (real-time for driving) Accuracy: 95% (2% loss acceptable for safety redundancy)

Deployed to: 4 million Teslas

Updates: Automatically download improved models weekly

Impact:

- Can run advanced AI on limited hardware
- Faster inference = safer driving (faster reaction time)
- Reduced latency = better driver experience

PATTERNS ACROSS ALL FIVE:

1. Trade efficiency for capability - Netflix: Explore sometimes, exploit usually (get better recommendations) - Google: Update incrementally, not from scratch (respond faster) - Apple: Train on-device not server (preserve privacy) - LinkedIn: Add noise to training (privacy guarantee) - Tesla: Compress model (fit on hardware)

2. Solve real business problems

- Netflix: User engagement and retention
- Google: Advertiser ROI - Apple: Privacy advantage - LinkedIn: Regulatory compliance - Tesla: Hardware constraints

3. Work at massive scale

- Netflix: 250M users - Google: Trillions of impressions - Apple: Billions of devices
- LinkedIn: 900M users
- Tesla: 4M cars

4. Measure impact carefully

- Each technique's impact measured quantitatively - A/B tested before rollout - Monitored continuously

post-rollout

DELIVERY TIPS

Real companies, real scale, real impact. • Each example shows technique + business problem it solves. • Netflix and Google examples show recommendation/optimization. • Apple and LinkedIn show privacy. • Tesla shows hardware constraints. • Patterns section ties all 5 together.

11. CHOOSING THE RIGHT TECHNIQUE

~5 minutes

WHAT'S ON SCREEN

Decision framework • Tradeoffs • When to use which • Combination strategies

SPEAKING SCRIPT

Not every problem needs every technique. Choose based on constraints and goals.

DECISION FRAMEWORK:

Question 1: What's your primary constraint?

[] Cost (too expensive to run): Answer: Model distillation, quantization, or online learning
Why: Reduce infrastructure cost

[] Latency (too slow):
Answer: Model distillation, quantization, edge deployment
Why: Smaller models run faster

[] Privacy (sensitive data):
Answer: Federated learning, differential privacy
Why: Protect user data

[] Accuracy (not good enough):
Answer: Online learning, multi-armed bandits
Why: Continuously improve or explore better options

Question 2: What's your data situation?

[] Streaming (data arrives continuously):
Answer: Online learning
Why: Handle drift, adapt in real-time

[] Distributed (data on multiple devices/companies):
Answer: Federated learning
Why: Learn without centralizing

[] Sensitive (privacy concerns):
Answer: Differential privacy, federated learning
Why: Protect sensitive information

[] Static (data doesn't change much):
Answer: Model distillation, quantization
Why: Optimize existing model

Question 3: What's your infrastructure?

[] Cloud servers (unlimited compute):
Any technique works. Choose based on other factors.

[] Limited devices (phones, edge):
Answer: Model distillation, quantization, pruning
Why: Fit on device, run fast

[] Real-time constraint (< 100ms):
Answer: Quantization, edge deployment
Why: Reduce latency

TECHNIQUE COMBINATIONS:

Most production systems use COMBINATION:

Large-scale recommendation system (Netflix-like):

- Online learning: Adapt to user preferences continuously
- Multi-armed bandits: Balance exploration and exploitation
- Result: Personalized recommendations that improve over time

On-device AI (Apple-like):

- Model distillation: Create small model
- Quantization: Reduce to INT8
- Federated learning: Train on-device without sending data
- Differential privacy: Privacy guarantee even if breached
- Result: Tiny, fast, private model on phone

High-scale cloud system (Google-like):

- Online learning: Continuous model updates
- A/B testing (multi-armed bandits): Test changes safely
- Model distillation: Create mobile version
- Quantization: Reduce cost
- Result: Fast, accurate, cost-effective at massive scale

HIRING AI EXAMPLE:

Requirement 1: Cost must be low (10K recommendations/day).

Requirement 2: Must personalize by company type.

Requirement 3: Some data is sensitive (success metrics private).

Solution:

- Online learning: Update model as hiring feedback arrives (improve over time)
- Federated approach: Companies train locally on their data, share updates (privacy)■r/>- Quantization: Deploy efficiently (reduce inference cost)■r/>- A/B testing: Explore new candidate profiles (multi-armed bandits)
- Result: Cost-effective, personalized, growing recommendations

WHEN TO AVOID:

Don't use online learning if: Data doesn't stream, or concept drift isn't an issue.

Don't use federated learning if: Data not distributed, privacy not required.

Don't use differential privacy if: Data not sensitive, regulatory not required.

Don't use distillation if: Accuracy loss unacceptable (e.g., medical diagnosis).

Don't use quantization if: Full precision required (e.g., scientific computing).

START SIMPLE:

Typical progression:

1. Build system that works (may be slow, expensive)
2. Identify bottleneck (what's limiting?)
3. Apply specific technique to fix bottleneck
4. Repeat as system scales

Example:

- Month 1: Build hiring AI, works but slow (500ms latency)■r/>- Month 2: Identify latency bottleneck (model inference)■r/>- Month 3: Distill model (500ms → 50ms)■r/>- Month 4: System fast enough■r/>- Month 6: As traffic grows, cost becomes issue■r/>- Month 7: Quantize model (30% cost reduction)■r/>- Month 9: Some hiring edge cases fail (private hiring data)■r/>- Month 10: Add differential privacy■r/>- Month 12: System is advanced, optimized, private

DELIVERY TIPS

Decision framework is practical. Students can use this. • Be honest: not every problem needs every technique. • Combinations show sophistication. • Start simple progression is realistic. • Hiring AI example ties to their project.

12. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Implement Optimization Technique' • Steps: Choose technique, Design implementation, Measure impact, Analyze tradeoffs

SPEAKING SCRIPT

Alright, lab time. You'll implement or design an advanced technique for your system.

YOUR TASK: Choose one technique (bandits, online learning, federated, privacy, distillation, quantization). Design how you'd implement it. Measure expected impact.

STEP 1: CHOOSE TECHNIQUE (1 min)

Which technique solves biggest problem in your hiring AI?

STEP 2: DESIGN IMPLEMENTATION (3 min)

How specifically would you implement it?

What data would you need?

What's the process?

STEP 3: MEASURE IMPACT (2 min)

What metric improves?

By how much?

What's the cost?

STEP 4: ANALYZE TRADEOFFS (2 min)

What do you gain?

What do you lose?

Is it worth it?

STEP 5: IMPLEMENTATION PLAN (2 min)

Timeline?

Resources needed?

Risks?

STEP 6: COMPARISON (2 min)

How does technique compare to alternatives?

Why did you choose this vs others?

DELIVERABLES:

1. Technique choice with justification
2. Implementation design document
3. Impact measurements (before/after)
4. Tradeoff analysis
5. Implementation timeline
6. Comparison to alternatives

DELIVERY TIPS

Lab applies techniques to their hiring system. • Students choose based on their constraints. • Implementation planning is realistic. • Impact measurement shows ROI calculation.

13. KEY TAKEAWAYS FROM SESSION 9

~5 minutes

WHAT'S ON SCREEN

6 Core Insights • Optimization is practical, Advanced matters at scale, Tradeoffs are real, Techniques combine

SPEAKING SCRIPT

Takeaway One: Optimization Is Practical Business Engineering

Not academic. Netflix uses bandits to get 5% engagement improvement = millions in revenue.

Takeaway Two: Advanced Techniques Address Real Constraints

Cost too high? Distillation. Latency too high? Quantization. Privacy risk? Federated + differential.

Takeaway Three: Trade-offs Are Fundamental

Less size = less accuracy. More privacy = more noise. More exploration = worse short-term performance.

Takeaway Four: Techniques Combine for Power

One technique good. Multiple techniques together? Transformational. 20x model compression possible.

Takeaway Five: Measure, Then Optimize

Don't guess what's expensive. Measure. Identify bottleneck. Optimize that specifically.

Takeaway Six: Privacy and Efficiency Are Increasingly Important

Federated learning and differential privacy aren't nice-to-have. They're requirements for modern systems.

DELIVERY TIPS

These six principles summarize Session 9. • Emphasis on practical business impact. • Tradeoffs are honest—optimization isn't free. • Combinations show sophistication. • Privacy shift reflects modern requirements.

14. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 10: Safety & Alignment' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 10 is where we address the hardest question: How do we ensure AI systems do what we want, not what we don't?

You've built systems, evaluated them, scaled them, and optimized them. But they're still powerful tools. What if they're used incorrectly? What if they discriminate? What if they harm users?

You'll learn: AI safety, bias detection, fairness, alignment, red teaming, and more.

This is the frontier of AI engineering: building not just powerful systems, but safe, fair, aligned systems.

Thank you all for today. You now understand advanced optimization techniques used by leading companies. You can build not just functional AI systems, but efficient, private, optimized systems at massive scale. See you in Session 10.

DELIVERY TIPS

Session 10 natural progression: Optimize what you've built → ensure it's safe. • Safety and alignment are future of AI engineering. • End on note of responsibility.